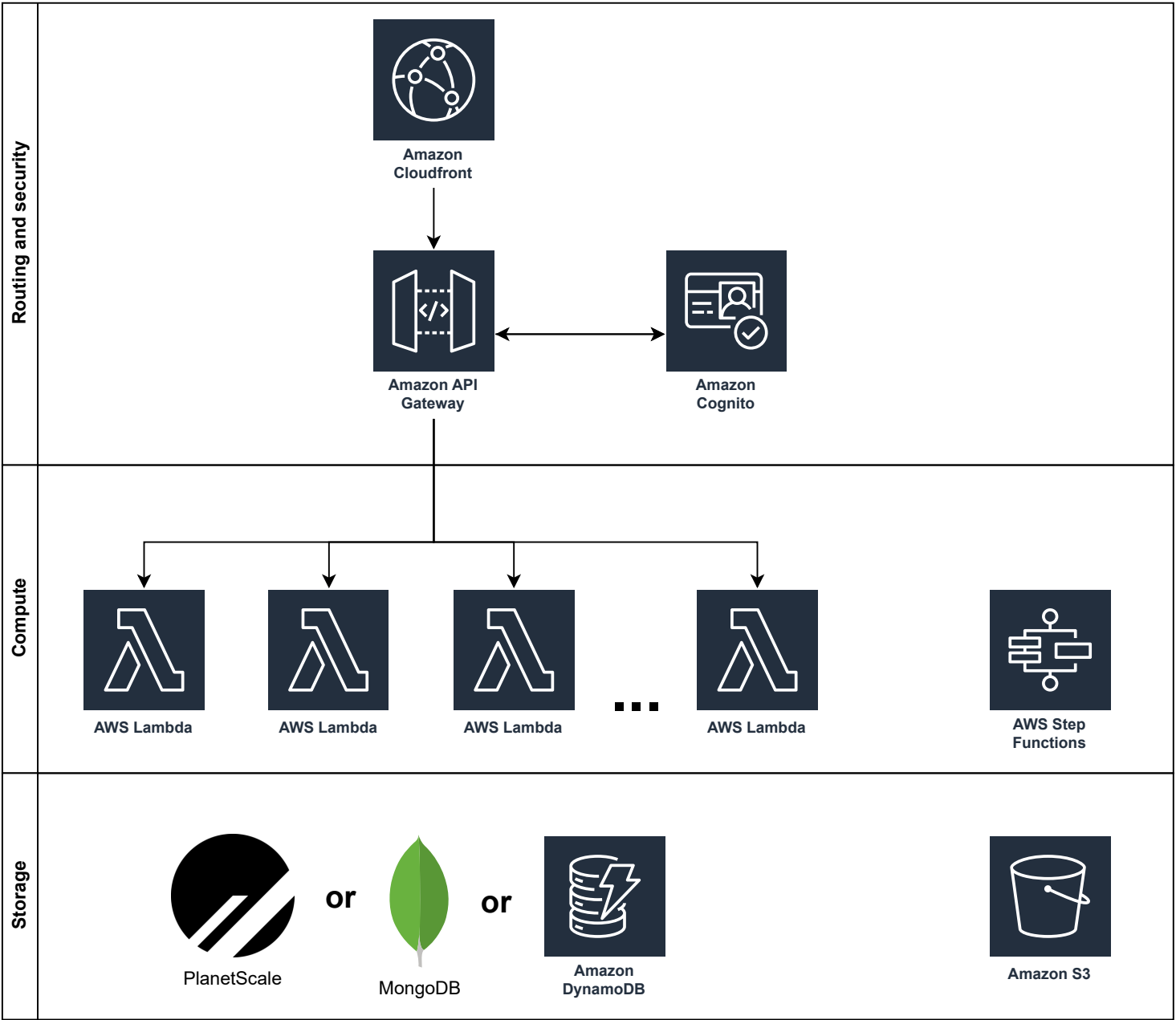


Data API



Overview

Common "REST API" setup in AWS using API Gateway API + Lambdas.

- Cloudfront serves as an (optional) caching layer
- Each lambda will handle a single verb+path combination (ie: `GET /users` and `POST /users` would be two different lambdas)
- Step Functions can be used to orchestrate long-running processes that require manual approvals or coordination between systems (such as managing payments or disitruptions)
- S3 will store any type of files that are uploaded by users (ie: pictures of tickets) or any files the system needs to generate (ie: invoices or agreements)
- The data layer is TBD based on the type of data we need to handle (not clear if things will be highly relational or more stand-alone-ish)
 - For our initial launch, the free tier for both MongoDB and PlanetScale would be enough
 - And DynamoDB is pay-per-utilization

This architecture allows for virtually infinite scalability (in terms of traffic or requests-per-second) while keeping the financial investment low (if there is no utilization, there are no charges). The downside is that it adds a tiny bit of overhead to the development process "traditional" frameworks like Express, Koa, etc. can't be used. But this tradeoff is usually well worth in.

Suggested tools and technologies

- All Lambdas related to the data API should be written in TypeScript
- Use either Serverless or SST to manage the infrastructure orchestration
- From a data perspective:
 - If we go the SQL route, then either Sequelize or Knex should be used
 - If we go the MongoDB route, then Mongoose is the way to go
 - If we go the DynamoDB route, a custom wrapper over the AWS library is recommended
- Custom domain registration and SSL certs should also all be managed via AWS
- We should ave at least three environments: dev, QA, and production
 - to start, we can get by by with dev and QA